Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

За шестой семестр

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:
Студент 3 курса
Группы ИИ-26(1)
Вирко Е.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провести **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с **фиксированным** шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с **адаптивным** шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:

- Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
  - MSE + фиксированный шаг (данные из ЛР №1);
  - MSE + адаптивный шаг (данные из ЛР №2);
  - BCE + фиксированный шаг;
  - BCE + адаптивный шаг.
- Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:

- Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
- Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
- Для сравнения также показать линию из ЛР №1 (если она отличается). **Описать** сравнение двух линий.

6. Реализовать режим функционирования:

- Пользователь вводит координаты x1,x2.
- Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
- Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Вариант 1

x_1	x_2	e
2	4	0
-2	4	0
2	-4	1
-2	-4	1

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import customtkinter as ctk
from functools import wraps

def require_training_data(func):
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        if self.inputs is None or self.labels is None:
            raise RuntimeError("Training data not loaded")
        return func(self, *args, **kwargs)
    return wrapper

class Perceptron:
    def __init__(self, in_size=0, step=0.1, tol=1e-2):
        self.weights = np.random.uniform(-1.0, 1.0, in_size + 1)
        self.lr = step
        self.eps = tol
        self.inputs = None
        self.labels = None

    def load_data(self, X):
        X = np.array(X)
        match X.ndim:
            case 1:
                X = X.reshape(1, -1)
            case 2:
                pass
            case _:
                print("Data must be 1D or 2D")
```

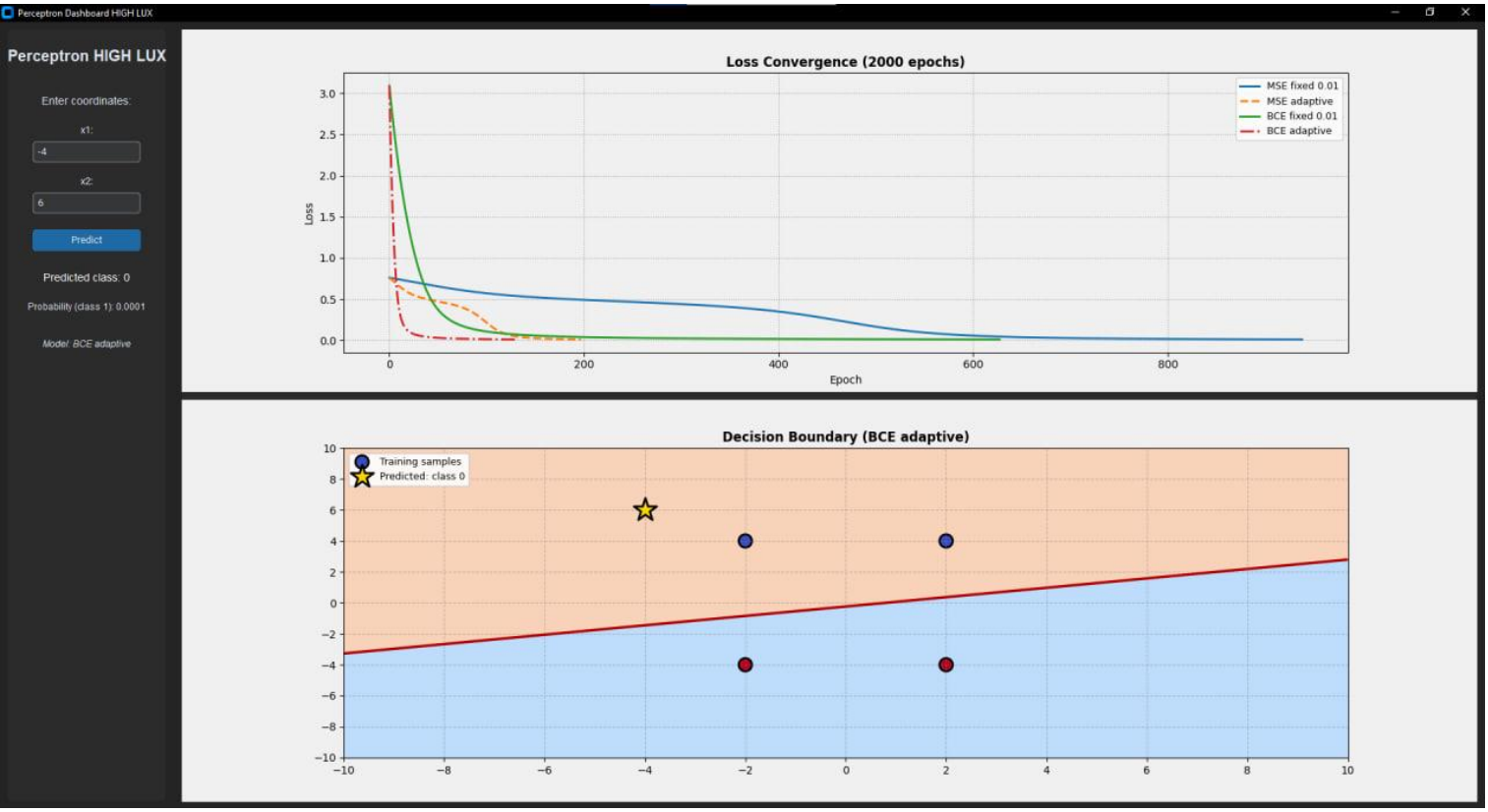
```
            return
        self.inputs = np.insert(X, 0, -1, axis=1)

    def load_labels(self, t):
        if self.inputs is None:
            print("Load data first")
            return
        if len(t) != len(self.inputs):
            print(f"Label count mismatch: expected {len(self.inputs)}, got {len(t)}")
            return
        self.labels = t

    def _raw(self, X_with_bias):
        return np.dot(X_with_bias, self.weights)

    _sigmoid = lambda self, z: 1.0 / (1.0 + np.exp(-z))

    def forward(self, X=None):
        if X is None:
            if self.inputs is None:
                raise RuntimeError("No input data provided and no training data loaded")
            data = self.inputs
        else:
            X = np.atleast_2d(X)
            data = np.insert(X, 0, -1, axis=1)
        return self._sigmoid(self._raw(data))

    def predict(self, X=None):
        return (self.forward(X) > 0.5).astype(int)

    def _mse_grad(self, y):
        err = y - self.labels
        deriv = y * (1 - y)
        self.weights -= self.lr * np.dot(err * deriv, self.inputs) / len(err)

    def _bce_grad(self, y):
        err = y - self.labels
        self.weights -= self.lr * np.dot(err, self.inputs) / len(err)

    mse_loss = lambda self, y: np.mean((y - self.labels) ** 2)
    bce_loss = lambda self, y: -np.mean(self.labels * np.log(y + 1e-12) + (1 - self.labels) * np.log(1 - y + 1e-12))

    @require_training_data
    def train_mse_fixed(self, epochs=500):
        hist = []
        for _ in range(epochs):
            y = self.forward()
            loss = self.mse_loss(y)
            hist.append(loss)
            if loss <= self.eps:
                break
            self._mse_grad(y)
        return hist

    @require_training_data
    def train_mse_adaptive(self, epochs=500):
        self.lr = 1.0 / np.mean(np.sum(self.inputs ** 2, axis=1))
        hist = []
        for _ in range(epochs):
            y = self.forward()
            loss = self.mse_loss(y)
            hist.append(loss)
            if loss <= self.eps:
                break
            self._mse_grad(y)
        return hist

    @require_training_data
```

```
    def train_bce_fixed(self, epochs=500):
        hist = []
        for _ in range(epochs):
            y = self.forward()
            loss = self.bce_loss(y)
            hist.append(loss)
            if loss <= self.eps:
                break
            self._bce_grad(y)
        return hist

    @require_training_data
    def train_bce_adaptive(self, epochs=500):
        self.lr = 1.0 / np.mean(np.sum(self.inputs ** 2, axis=1))
        hist = []
        for _ in range(epochs):
            y = self.forward()
            loss = self.bce_loss(y)
            hist.append(loss)
            if loss <= self.eps:
                break
            self._bce_grad(y)
        return hist

class PerceptronApp(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Perceptron Dashboard HIGH LUX")
        self.geometry("1300x700")
        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("blue")

        self.samples = np.array([[2, 4], [-2, 4], [2, -4], [-2, -4]])
        self.labels = np.array([0, 0, 1, 1])

        np.random.seed(42)
        init_w = np.random.uniform(0, 1.0, 3)

        self.model_fixed_mse = Perceptron(in_size=2, step=0.01)
        self.model_fixed_mse.weights = init_w.copy()
        self.model_fixed_mse.load_data(self.samples)
        self.model_fixed_mse.load_labels(self.labels)
        self.hist_fixed_mse = self.model_fixed_mse.train_mse_fixed(2000)

        self.model_adapt_mse = Perceptron(in_size=2)
        self.model_adapt_mse.weights = init_w.copy()
        self.model_adapt_mse.load_data(self.samples)
        self.model_adapt_mse.load_labels(self.labels)
        self.hist_adapt_mse = self.model_adapt_mse.train_mse_adaptive(2000)

        self.model_fixed_bce = Perceptron(in_size=2, step=0.01)
        self.model_fixed_bce.weights = init_w.copy()
        self.model_fixed_bce.load_data(self.samples)
        self.model_fixed_bce.load_labels(self.labels)
        self.hist_fixed_bce = self.model_fixed_bce.train_bce_fixed(2000)

        self.model_adapt_bce = Perceptron(in_size=2, step=0.01)
        self.model_adapt_bce.weights = init_w.copy()
        self.model_adapt_bce.load_data(self.samples)
        self.model_adapt_bce.load_labels(self.labels)
        self.hist_adapt_bce = self.model_adapt_bce.train_bce_adaptive(2000)


        self.model = self.model_adapt_bce


        self.left_frame = ctk.CTkFrame(self, width=250)
```

```
    self.left_frame.pack(side="left", fill="y", padx=10, pady=10)


    self.right_frame = ctk.CTkFrame(self)
    self.right_frame.pack(side="right", expand=True, fill="both", padx=10, pady=10)


    self.top_graph_frame = ctk.CTkFrame(self.right_frame)
    self.top_graph_frame.pack(side="top", expand=True, fill="both", pady=(0, 5))

    self.bottom_graph_frame = ctk.CTkFrame(self.right_frame)
    self.bottom_graph_frame.pack(side="bottom", expand=True, fill="both", pady=(5, 0))

    self.create_widgets()
    self.plot_loss_convergence()
    self.draw_decision_surface()

  def create_widgets(self):
    title = ctk.CTkLabel(self.left_frame, text="Perceptron HIGH LUX", font=("Arial", 20, "bold"))
    title.pack(pady=(20, 30))

    ctk.CTkLabel(self.left_frame, text="Enter coordinates:", font=("Arial", 14)).pack(pady=(0, 10))

    ctk.CTkLabel(self.left_frame, text="x1:").pack()
    self.entry_x1 = ctk.CTkEntry(self.left_frame, placeholder_text="e.g. 3")
    self.entry_x1.pack(pady=(0, 10))

    ctk.CTkLabel(self.left_frame, text="x2:").pack()
    self.entry_x2 = ctk.CTkEntry(self.left_frame, placeholder_text="e.g. -2")
    self.entry_x2.pack(pady=(0, 20))

    self.predict_btn = ctk.CTkButton(self.left_frame, text="Predict", command=self.on_predict)
    self.predict_btn.pack(pady=(0, 20))

    self.result_label = ctk.CTkLabel(self.left_frame, text="", font=("Arial", 14))
    self.result_label.pack(pady=(0, 10))

    self.prob_label = ctk.CTkLabel(self.left_frame, text="", font=("Arial", 12))
    self.prob_label.pack()

    model_info = ctk.CTkLabel(self.left_frame, text="Model: BCE adaptive", font=("Arial", 12, "italic"))
    model_info.pack(pady=(20, 0))

  def plot_loss_convergence(self):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.plot(self.hist_fixed_mse, label='MSE fixed 0.01', color='#1f77b4', linewidth=2)
    ax.plot(self.hist_adapt_mse, label='MSE adaptive', color='#ff7f0e', linestyle='--', linewidth=2)
    ax.plot(self.hist_fixed_bce, label='BCE fixed 0.01', color='#2ca02c', linewidth=2)
    ax.plot(self.hist_adapt_bce, label='BCE adaptive', color='#d62728', linestyle='-.', linewidth=2)

    ax.set_title('Loss Convergence (2000 epochs)', fontsize=12, fontweight='bold')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.grid(True, linestyle=':', alpha=0.7, color='gray')
    ax.legend(loc='upper right', fontsize=9)
    ax.set_facecolor('#f0f0f0')
    fig.patch.set_facecolor('#f0f0f0')

    if hasattr(self, 'canvas_loss'):
        self.canvas_loss.get_tk_widget().destroy()
    self.canvas_loss = FigureCanvasTkAgg(fig, master=self.top_graph_frame)
    self.canvas_loss.draw()
    self.canvas_loss.get_tk_widget().pack(expand=True, fill='both')
    plt.close(fig)

  def draw_decision_surface(self, user_pt=None, user_cls=None):
    fig, ax = plt.subplots(figsize=(6, 4.5))
    ax.set_xlim(-10, 10)
```

```
        ax.set_ylim(-10, 10)
        ax.set_xticks(range(-10, 11, 2))
        ax.set_yticks(range(-10, 11, 2))
        ax.grid(True, linestyle='--', alpha=0.4, color='gray')
        ax.set_facecolor('#eaeaf2')

        xx, yy = np.meshgrid(np.linspace(-10, 10, 300), np.linspace(-10, 10, 300))
        grid = np.c_[xx.ravel(), yy.ravel()]
        grid_with_bias = np.insert(grid, 0, -1, axis=1)
        Z_prob = self.model._sigmoid(self.model._raw(grid_with_bias)).reshape(xx.shape)
        Z_raw = self.model._raw(grid_with_bias).reshape(xx.shape)

        ax.contourf(xx, yy, Z_prob, levels=[-0.1, 0.5, 1.1], colors=['#fdcdac', '#b3d9ff'], alpha=0.8)
        ax.contour(xx, yy, Z_raw, levels=[0], colors='#b30000', linewidths=2.5, linestyles='solid')

        ax.scatter(self.samples[:, 0], self.samples[:, 1], c=self.labels,
                   cmap='coolwarm', edgecolors='black', s=150, linewidths=2, label='Training samples')

        if user_pt is not None:
            ax.scatter(*user_pt, color='#ffd700', marker='*', s=500, edgecolors='black', linewidth=2,
                       label=f'Predicted: class {user_cls}', zorder=10)

        ax.legend(loc='upper left', fontsize=9, framealpha=0.9)
        ax.set_title('Decision Boundary (BCE adaptive)', fontsize=12, fontweight='bold')
        fig.patch.set_facecolor('#f0f0f0')

        if hasattr(self, 'canvas_boundary'):
            self.canvas_boundary.get_tk_widget().destroy()
        self.canvas_boundary = FigureCanvasTkAgg(fig, master=self.bottom_graph_frame)
        self.canvas_boundary.draw()
        self.canvas_boundary.get_tk_widget().pack(expand=True, fill='both')
        plt.close(fig)

    def on_predict(self):
        try:
            x1 = float(self.entry_x1.get())
            x2 = float(self.entry_x2.get())
        except ValueError:
            self.result_label.configure(text="Invalid input", text_color="red")
            return

        pt = np.array([[x1, x2]])
        prob = self.model.forward(pt)[0]
        cls = self.model.predict(pt)[0]

        self.result_label.configure(text=f"Predicted class: {cls}", text_color="white")
        self.prob_label.configure(text=f"Probability (class 1): {prob:.4f}")

        self.draw_decision_surface(user_pt=(x1, x2), user_cls=cls)

if __name__ == "__main__":
    app = PerceptronApp()
    app.mainloop()
```

Результат:



Вывод: Изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с **сигмоидной** функцией активации и функцией потерь в виде **бинарной кросс-энтропии** (Binary Cross-Entropy, BCE). Провел **сравнительный анализ** скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).